Page 1: Operating Systems Fundamentals

An operating system is system software that manages computer hardware and provides common services for application programs. It acts as an intermediary between users, applications, and hardware. Important responsibilities include process management, memory management, file management, device management, protection, security, and resource allocation. The kernel is the core component that remains active while the system is running. It manages privileged operations and provides mechanisms through which applications request services. A user program normally cannot directly access hardware resources; instead, it uses system calls provided by the operating system. Examples of operating systems include Windows, Linux, macOS, Android, and many embedded operating systems. Operating systems can be classified in several ways, including batch systems, multiprogramming systems, time-sharing systems, distributed systems, network systems, and real-time systems. A batch system executes jobs with little or no interaction while a time-sharing system rapidly switches the processor among users or processes to provide interactive service. The main objective of an operating system is not simply to execute programs, but to use resources efficiently while providing convenient and reliable execution. It must coordinate CPU time, memory space, storage, input/output devices, and communication mechanisms. The operating system also handles errors, enforces protection boundaries, and keeps applications from interfering with one another. Modern systems use layers of abstraction so that programmers can work with processes, files, sockets, virtual memory, and other logical resources without knowing the exact hardware implementation. This abstraction is one of the most important ideas in operating-system design. An operating system is system software that manages computer hardware and provides common services for application programs. It acts as an intermediary between users, applications, and hardware. Important responsibilities include process management, memory management, file management, device management, protection, security, and resource allocation. The kernel is the core component that remains active while the system is running. It manages privileged operations and provides mechanisms through which applications request services. A user program normally cannot directly access hardware resources; instead, it uses system calls provided by the operating system. Examples of operating systems include Windows, Linux, macOS, Android, and many embedded operating systems. Operating systems can be classified in several ways, including batch systems, multiprogramming systems, time-sharing systems, distributed systems, network systems, and real-time systems.

A batch system executes jobs with little or no interaction while a time-sharing system rapidly switches the processor among users or processes to provide interactive service. The main objective of an operating system is not simply to execute programs, but to use resources efficiently while providing convenient and reliable execution. It must coordinate CPU time, memory space, storage, input/output devices, and communication mechanisms. The operating system also handles errors, enforces protection boundaries, and keeps applications from interfering with one another. Modern systems use layers of abstraction so that programmers can work with processes, files, sockets, virtual memory, and other logical resources without knowing the exact hardware implementation. This abstraction is one of the most important ideas in operating-system design.

Page 2: Processes and Process States

A process is a program in execution. It includes the program code, current activity, CPU registers, stack, data, and other execution-related information. The operating system represents a process using a process control block, or PCB. The PCB can contain the process identifier, process state, program counter, CPU registers, scheduling information, memory-management information, accounting information, and I/O status. A process normally moves through states such as new, ready, running, waiting or blocked, and terminated. In the ready state, a process is prepared to execute but is waiting for CPU allocation. In the running state, its instructions are currently being executed. A process may enter the waiting state when it needs an event such as completion of an input/output operation. After the event occurs, it can return to the ready state. The scheduler selects processes from the ready queue according to a scheduling policy. A context switch occurs when the CPU changes from one process to another. During a context switch, the operating system saves the state of the current process and restores the state of another process. Context switching is necessary for multitasking but adds overhead because useful application instructions are not executed during the switch itself. Processes can also create other processes. The relationship may form a process tree. Interprocess communication allows processes to exchange data or synchronize their activities. Common IPC mechanisms include pipes, shared memory, message passing, sockets, and signals. Correct process management is essential for responsiveness, resource sharing, and system stability. A process is a program in execution. It includes the program code, current activity, CPU registers, stack, data, and other execution-related information. The operating system represents a process using a process control block, or PCB. The PCB can contain the process identifier, process state, program counter, CPU registers, scheduling information, memory-management information, accounting information, and I/O status. A process normally moves through states such as new, ready, running, waiting or blocked, and terminated. In the ready state, a process is prepared to execute but is waiting for CPU allocation. In the running state, its instructions are currently being executed. A process may enter the waiting state when it needs an event such as completion of an input/output operation. After the event occurs, it can return to the ready state. The scheduler selects processes from the ready queue according to a scheduling policy. A context switch occurs when the CPU changes from one process to another. During a context switch, the operating system saves the state of the current process and restores the state of another process.

Context switching is necessary for multitasking but adds overhead because useful application instructions are not executed during the switch itself. Processes can also create other processes. The relationship may form a process tree. Interprocess communication allows processes to exchange data or synchronize their activities. Common IPC mechanisms include pipes, shared memory, message passing, sockets, and signals. Correct process management is essential for responsiveness, resource sharing, and system stability.

Page 3: CPU Scheduling

CPU scheduling determines which ready process should receive the processor. The goal of scheduling depends on system requirements. Common performance measures include CPU utilization, throughput, turnaround time, waiting time, response time, and fairness. First-Come, First-Served scheduling executes processes in order of arrival. It is simple but can cause the convoy effect when a long process delays many shorter processes. Shortest Job First selects the process with the smallest expected CPU burst. It can minimize average waiting time under ideal assumptions, but exact future burst lengths are generally unknown. Shortest Remaining Time First is the preemptive form of shortest-job scheduling. Priority scheduling selects processes according to priority and can be preemptive or non-preemptive. A low-priority process may suffer starvation if higher-priority processes continue to arrive. Aging gradually increases the priority of waiting processes and can reduce starvation. Round Robin scheduling assigns each process a time quantum. After the quantum expires, a running process can be preempted and placed at the end of the ready queue. A small quantum improves responsiveness but can increase context-switch overhead. A very large quantum makes Round Robin behave more like FCFS. Multilevel queue scheduling separates processes into different queues based on characteristics such as foreground and background work. Multilevel feedback queue scheduling allows processes to move between queues according to observed behavior. Scheduling algorithms represent trade-offs rather than a single universally optimal method. Interactive systems generally emphasize response time, while batch workloads may emphasize throughput and turnaround time. CPU scheduling determines which ready process should receive the processor. The goal of scheduling depends on system requirements. Common performance measures include CPU utilization, throughput, turnaround time, waiting time, response time, and fairness. First-Come, First-Served scheduling executes processes in order of arrival. It is simple but can cause the convoy effect when a long process delays many shorter processes. Shortest Job First selects the process with the smallest expected CPU burst. It can minimize average waiting time under ideal assumptions, but exact future burst lengths are generally unknown. Shortest Remaining Time First is the preemptive form of shortest-job scheduling. Priority scheduling selects processes according to priority and can be preemptive or non-preemptive. A low-priority process may suffer starvation if higher-priority processes continue to arrive. Aging gradually increases the priority of waiting processes and can reduce starvation. Round Robin scheduling assigns each process a time quantum. After the quantum expires, a running process can be preempted and placed at the end of the ready queue.

A small quantum improves responsiveness but can increase context-switch overhead. A very large quantum makes Round Robin behave more like FCFS. Multilevel queue scheduling separates processes into different queues based on characteristics such as foreground and background work. Multilevel feedback queue scheduling allows processes to move between queues according to observed behavior. Scheduling algorithms represent trade-offs rather than a single universally optimal method. Interactive systems generally emphasize response time, while batch workloads may emphasize throughput and turnaround time.

Page 4: Threads and Multithreading

A thread is the smallest unit of CPU execution within a process. A process can contain one or multiple threads. Threads belonging to the same process normally share code, data, and operating-system resources, while each thread has its own program counter, register set, and stack. Multithreading can improve responsiveness because one thread can continue useful work while another waits for I/O. It can also improve resource utilization and allow parallel execution on multicore processors. User-level threads are managed by a user-space library, while kernel-level threads are managed by the operating system. User-level thread operations can be fast, but certain blocking operations may affect the entire process depending on the threading model. Kernel threads provide operating-system awareness and can be scheduled independently. Multithreading introduces synchronization challenges because shared data can be accessed concurrently. A race condition occurs when the result depends on the timing or ordering of concurrent operations. Critical sections are code regions that access shared resources and therefore require suitable synchronization. Mutexes, semaphores, monitors, and atomic operations are common synchronization mechanisms. A mutex normally provides mutual exclusion so that only one thread enters a protected region at a time. A semaphore maintains a counter and can be used for signaling and resource control. Good concurrent design requires identifying shared state, protecting critical sections, avoiding unnecessary locking, and considering deadlock and starvation. Multithreading is widely used in servers, browsers, operating systems, databases, games, and scientific applications. A thread is the smallest unit of CPU execution within a process. A process can contain one or multiple threads. Threads belonging to the same process normally share code, data, and operating-system resources, while each thread has its own program counter, register set, and stack. Multithreading can improve responsiveness because one thread can continue useful work while another waits for I/O. It can also improve resource utilization and allow parallel execution on multicore processors. User-level threads are managed by a user-space library, while kernel-level threads are managed by the operating system. User-level thread operations can be fast, but certain blocking operations may affect the entire process depending on the threading model. Kernel threads provide operating-system awareness and can be scheduled independently. Multithreading introduces synchronization challenges because shared data can be accessed concurrently. A race condition occurs when the result depends on the timing or ordering of concurrent operations. Critical sections are code regions that access shared resources and therefore require suitable synchronization.

Mutexes, semaphores, monitors, and atomic operations are common synchronization mechanisms. A mutex normally provides mutual exclusion so that only one thread enters a protected region at a time. A semaphore maintains a counter and can be used for signaling and resource control. Good concurrent design requires identifying shared state, protecting critical sections, avoiding unnecessary locking, and considering deadlock and starvation. Multithreading is widely used in servers, browsers, operating systems, databases, games, and scientific applications.

Page 5: Process Synchronization

Process synchronization coordinates concurrent processes or threads so that shared resources are accessed safely. The critical-section problem requires a solution satisfying mutual exclusion, progress, and bounded waiting. Mutual exclusion means that at most one process can execute the critical section for a particular shared resource at a time. Progress means that if no process is in the critical section and some processes want to enter, the decision about who enters should not be postponed indefinitely. Bounded waiting means that a process should not wait forever after requesting entry. Hardware instructions such as test-and-set and compare-and-swap can support synchronization primitives. Semaphores provide another important mechanism. A binary semaphore can represent a lock-like resource, while a counting semaphore can represent multiple identical resources. The classic producer-consumer problem illustrates synchronization when one process produces items and another consumes them through a bounded buffer. The readers-writers problem studies situations where multiple readers can share a resource but writers require exclusive access. The dining philosophers problem demonstrates challenges involving resource allocation and deadlock. Synchronization mechanisms must be designed carefully because incorrect locking can cause deadlock, starvation, or performance problems. Busy waiting can waste CPU cycles when a process repeatedly checks a condition, while blocking synchronization allows the operating system to suspend a waiting process. Modern operating systems combine hardware support, atomic operations, scheduler behavior, and high-level synchronization abstractions to coordinate concurrent execution. Process synchronization coordinates concurrent processes or threads so that shared resources are accessed safely. The critical-section problem requires a solution satisfying mutual exclusion, progress, and bounded waiting. Mutual exclusion means that at most one process can execute the critical section for a particular shared resource at a time. Progress means that if no process is in the critical section and some processes want to enter, the decision about who enters should not be postponed indefinitely. Bounded waiting means that a process should not wait forever after requesting entry. Hardware instructions such as test-and-set and compare-and-swap can support synchronization primitives. Semaphores provide another important mechanism. A binary semaphore can represent a lock-like resource, while a counting semaphore can represent multiple identical resources. The classic producer-consumer problem illustrates synchronization when one process produces items and another consumes them through a bounded buffer. The readers-writers problem studies situations where multiple readers can share a resource but writers require exclusive access.

The dining philosophers problem demonstrates challenges involving resource allocation and deadlock. Synchronization mechanisms must be designed carefully because incorrect locking can cause deadlock, starvation, or performance problems. Busy waiting can waste CPU cycles when a process repeatedly checks a condition, while blocking synchronization allows the operating system to suspend a waiting process. Modern operating systems combine hardware support, atomic operations, scheduler behavior, and high-level synchronization abstractions to coordinate concurrent execution.

Page 6: Deadlocks

A deadlock is a situation in which a set of processes is permanently blocked because every process is waiting for a resource or event that can only be produced by another process in the same set. Four necessary conditions characterize deadlock: mutual exclusion, hold and wait, no preemption, and circular wait. Mutual exclusion means that at least one resource is non-shareable. Hold and wait means a process holds at least one resource while waiting for another. No preemption means resources cannot simply be taken away from a process. Circular wait means there is a cycle of processes where each process waits for a resource held by the next process. Operating systems can handle deadlocks through prevention, avoidance, detection and recovery, or by ignoring the issue when the cost of handling it is considered too high. Deadlock prevention eliminates one of the necessary conditions. Deadlock avoidance makes allocation decisions only when the resulting state remains safe. The Banker's algorithm is a well-known avoidance algorithm that uses maximum resource demands and available resources to determine whether granting a request can keep the system in a safe state. Deadlock detection allows deadlocks to occur and periodically checks for them. Recovery may involve terminating processes or preempting resources. A safe state is not necessarily a state with no waiting; rather, it is a state from which there exists an ordering in which all processes can finish. Deadlock and starvation are different concepts. Starvation occurs when a process waits indefinitely because scheduling or resource allocation repeatedly favors other processes. Careful resource ordering and appropriate synchronization can reduce these problems. A deadlock is a situation in which a set of processes is permanently blocked because every process is waiting for a resource or event that can only be produced by another process in the same set. Four necessary conditions characterize deadlock: mutual exclusion, hold and wait, no preemption, and circular wait. Mutual exclusion means that at least one resource is non-shareable. Hold and wait means a process holds at least one resource while waiting for another. No preemption means resources cannot simply be taken away from a process. Circular wait means there is a cycle of processes where each process waits for a resource held by the next process. Operating systems can handle deadlocks through prevention, avoidance, detection and recovery, or by ignoring the issue when the cost of handling it is considered too high. Deadlock prevention eliminates one of the necessary conditions. Deadlock avoidance makes allocation decisions only when the resulting state remains safe.

The Banker's algorithm is a well-known avoidance algorithm that uses maximum resource demands and available resources to determine whether granting a request can keep the system in a safe state. Deadlock detection allows deadlocks to occur and periodically checks for them. Recovery may involve terminating processes or preempting resources. A safe state is not necessarily a state with no waiting; rather, it is a state from which there exists an ordering in which all processes can finish. Deadlock and starvation are different concepts. Starvation occurs when a process waits indefinitely because scheduling or resource allocation repeatedly favors other processes. Careful resource ordering and appropriate synchronization can reduce these problems.

Page 7: Memory Management

Memory management controls the allocation and deallocation of main memory among processes. The operating system must keep track of which memory regions are available and which are occupied. Early systems used contiguous allocation, where each process occupied one continuous memory region. Fixed partitioning can lead to internal fragmentation, while variable partitioning can lead to external fragmentation. Paging divides physical memory into fixed-size frames and logical memory into fixed-size pages. A page can be placed into any available frame, removing the need for a process to occupy contiguous physical memory. A page table maps virtual page numbers to physical frame numbers. A translation lookaside buffer, or TLB, is a fast cache used to speed up address translation. Segmentation divides a program into logical segments such as code, data, stack, and procedures. Segments may have different sizes and protection attributes. Some architectures and operating systems combine paging and segmentation concepts. Memory protection prevents one process from accessing memory belonging to another process without permission. Dynamic memory allocation requires efficient tracking of free and occupied regions. Fragmentation is an important concern. Internal fragmentation is wasted space inside allocated blocks, while external fragmentation is free space scattered between allocated blocks. Compaction can reduce external fragmentation by moving allocated regions, but it is expensive. Virtual memory extends the apparent memory available to programs by using secondary storage as a backing store. It allows processes to execute even when not all of their pages are simultaneously resident in physical memory. Effective memory management improves utilization, isolation, performance, and system reliability. Memory management controls the allocation and deallocation of main memory among processes. The operating system must keep track of which memory regions are available and which are occupied. Early systems used contiguous allocation, where each process occupied one continuous memory region. Fixed partitioning can lead to internal fragmentation, while variable partitioning can lead to external fragmentation. Paging divides physical memory into fixed-size frames and logical memory into fixed-size pages. A page can be placed into any available frame, removing the need for a process to occupy contiguous physical memory. A page table maps virtual page numbers to physical frame numbers. A translation lookaside buffer, or TLB, is a fast cache used to speed up address translation. Segmentation divides a program into logical segments such as code, data, stack, and procedures. Segments may have different sizes and protection attributes. Some architectures and operating systems combine paging and segmentation concepts.

Memory protection prevents one process from accessing memory belonging to another process without permission. Dynamic memory allocation requires efficient tracking of free and occupied regions. Fragmentation is an important concern. Internal fragmentation is wasted space inside allocated blocks, while external fragmentation is free space scattered between allocated blocks. Compaction can reduce external fragmentation by moving allocated regions, but it is expensive. Virtual memory extends the apparent memory available to programs by using secondary storage as a backing store. It allows processes to execute even when not all of their pages are simultaneously resident in physical memory. Effective memory management improves utilization, isolation, performance, and system reliability.

Page 8: Virtual Memory and Page Replacement

Virtual memory allows a process to use a logical address space that can be larger than the available physical memory. Demand paging loads a page into physical memory only when it is needed. If a referenced page is not present, a page fault occurs. The operating system handles the fault by locating the page on secondary storage, selecting a free frame or a victim page, loading the required page, updating page-table information, and restarting the interrupted instruction. Page replacement becomes necessary when no free frame is available. FIFO replaces the page that has been in memory for the longest time. It is simple but can suffer from Belady's anomaly, where increasing the number of frames can sometimes increase the number of page faults. Optimal page replacement removes the page whose next use is farthest in the future. It provides the minimum possible page faults for a known reference string, but future references are not available to a real operating system. LRU replaces the page that has not been used for the longest period. It approximates the locality behavior of programs and is generally more practical than the optimal algorithm. Other approaches include second-chance and clock algorithms. The working-set concept attempts to keep the actively used pages in memory. If a system spends most of its time servicing page faults instead of executing useful instructions, it may enter a condition called thrashing. Increasing memory, improving locality, or adjusting the degree of multiprogramming can help reduce thrashing. Page replacement is an important part of virtual-memory performance because disk or SSD access is much slower than normal memory access. Virtual memory allows a process to use a logical address space that can be larger than the available physical memory. Demand paging loads a page into physical memory only when it is needed. If a referenced page is not present, a page fault occurs. The operating system handles the fault by locating the page on secondary storage, selecting a free frame or a victim page, loading the required page, updating page-table information, and restarting the interrupted instruction. Page replacement becomes necessary when no free frame is available. FIFO replaces the page that has been in memory for the longest time. It is simple but can suffer from Belady's anomaly, where increasing the number of frames can sometimes increase the number of page faults. Optimal page replacement removes the page whose next use is farthest in the future. It provides the minimum possible page faults for a known reference string, but future references are not available to a real operating system. LRU replaces the page that has not been used for the longest period. It approximates the locality behavior of programs and is generally more practical than the optimal algorithm.

Other approaches include second-chance and clock algorithms. The working-set concept attempts to keep the actively used pages in memory. If a system spends most of its time servicing page faults instead of executing useful instructions, it may enter a condition called thrashing. Increasing memory, improving locality, or adjusting the degree of multiprogramming can help reduce thrashing. Page replacement is an important part of virtual-memory performance because disk or SSD access is much slower than normal memory access.

Page 9: File Systems

A file system organizes data on secondary storage and provides abstractions for creating, naming, reading, writing, and deleting files. A file is a named collection of related information stored on persistent storage. Common file attributes include name, type, size, location, protection information, timestamps, and ownership. Operating systems provide file operations such as create, open, read, write, seek, close, and delete. Directories organize files into a hierarchical structure. A path identifies the location of a file within the directory hierarchy. File allocation methods determine how file blocks are placed on storage. Contiguous allocation stores a file in consecutive blocks and offers efficient sequential and direct access, but it can suffer from external fragmentation and difficulty growing files. Linked allocation stores each file as a linked list of blocks, avoiding external fragmentation but making direct access slower. Indexed allocation uses an index block containing pointers to file blocks and supports direct access without requiring contiguous storage. Modern file systems use more advanced metadata structures and journaling techniques. Journaling records intended changes so the file system can recover more safely after a crash. File protection controls who can read, write, execute, or modify files. Access-control mechanisms may use user identities, groups, permissions, access-control lists, or capabilities. Storage systems also use buffering, caching, and scheduling to improve performance. File-system reliability depends on correct metadata updates, consistency, recovery procedures, and protection against corruption. The operating system provides a uniform interface so applications can work with files without needing to understand the physical layout of the storage device. A file system organizes data on secondary storage and provides abstractions for creating, naming, reading, writing, and deleting files. A file is a named collection of related information stored on persistent storage. Common file attributes include name, type, size, location, protection information, timestamps, and ownership. Operating systems provide file operations such as create, open, read, write, seek, close, and delete. Directories organize files into a hierarchical structure. A path identifies the location of a file within the directory hierarchy. File allocation methods determine how file blocks are placed on storage. Contiguous allocation stores a file in consecutive blocks and offers efficient sequential and direct access, but it can suffer from external fragmentation and difficulty growing files. Linked allocation stores each file as a linked list of blocks, avoiding external fragmentation but making direct access slower.

Indexed allocation uses an index block containing pointers to file blocks and supports direct access without requiring contiguous storage. Modern file systems use more advanced metadata structures and journaling techniques. Journaling records intended changes so the file system can recover more safely after a crash. File protection controls who can read, write, execute, or modify files. Access-control mechanisms may use user identities, groups, permissions, access-control lists, or capabilities. Storage systems also use buffering, caching, and scheduling to improve performance. File-system reliability depends on correct metadata updates, consistency, recovery procedures, and protection against corruption. The operating system provides a uniform interface so applications can work with files without needing to understand the physical layout of the storage device.

Page 10: I/O Systems, Security, and System Calls

Input/output management allows the operating system to communicate with devices such as keyboards, disks, displays, network interfaces, and printers. Device drivers provide software interfaces between the operating system and hardware devices. Because devices have different characteristics, the operating system uses abstraction mechanisms to present consistent interfaces to applications. Polling repeatedly checks whether a device is ready, while interrupts allow a device to notify the processor when an event occurs. Direct memory access, or DMA, allows certain devices to transfer data directly to memory with limited CPU involvement, improving efficiency for large transfers. Buffering temporarily stores data during transfers, while caching keeps frequently used information available for faster access. Spooling places jobs into a queue for devices such as printers. System calls provide the controlled interface through which application programs request operating-system services. Examples include process creation, file operations, memory allocation, and communication operations. Security protects system resources from unauthorized access and malicious behavior. Authentication establishes the identity of a user or service, while authorization determines what that identity is allowed to access. Least privilege means granting only the permissions required to perform a task. Operating systems also use isolation, access control, encryption support, secure boot mechanisms, logging, and auditing. A secure design considers both hardware and software threats. Protection mechanisms are enforced at appropriate privilege levels so that ordinary applications cannot bypass them. Reliability and security are closely related because failures in device handling, memory protection, authentication, or file permissions can affect the entire system. Modern operating systems therefore integrate resource management, protection, security, and recovery mechanisms into a single platform. Input/output management allows the operating system to communicate with devices such as keyboards, disks, displays, network interfaces, and printers. Device drivers provide software interfaces between the operating system and hardware devices. Because devices have different characteristics, the operating system uses abstraction mechanisms to present consistent interfaces to applications. Polling repeatedly checks whether a device is ready, while interrupts allow a device to notify the processor when an event occurs. Direct memory access, or DMA, allows certain devices to transfer data directly to memory with limited CPU involvement, improving efficiency for large transfers. Buffering temporarily stores data during transfers, while caching keeps frequently used information available for faster access. Spooling places jobs into a queue for devices such as printers.

System calls provide the controlled interface through which application programs request operating-system services. Examples include process creation, file operations, memory allocation, and communication operations. Security protects system resources from unauthorized access and malicious behavior. Authentication establishes the identity of a user or service, while authorization determines what that identity is allowed to access. Least privilege means granting only the permissions required to perform a task. Operating systems also use isolation, access control, encryption support, secure boot mechanisms, logging, and auditing. A secure design considers both hardware and software threats. Protection mechanisms are enforced at appropriate privilege levels so that ordinary applications cannot bypass them. Reliability and security are closely related because failures in device handling, memory protection, authentication, or file permissions can affect the entire system. Modern operating systems therefore integrate resource management, protection, security, and recovery mechanisms into a single platform.